

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Proyecto: Base de datos de un restaurante

Profesor: Ing. Fernando Arreola Franco

Integrantes:

Cruz Ortega Mauricio

Gómez Domingo José Antonio

Hernández Muciño Angel Gabriel

Mugartegui Vaca Ricardo

Mayo 2026

Índice

Introducción	2
Plan de trabajo	2
Diseño	3
Diseño Conceptual: Modelo Entidad-Relación (MER)	3
Diseño Lógico: Modelo Relacional (MR)	4
Nota de Adaptación Arquitectural	4
Implementación	5
Lenguaje de Definición de Datos (DDL) - Creación de la Base de datos y tablas	5
Vistas computadas (Views)	9
Funciones (Functions)	10
Disparadores (Triggers)	11
Índice (Index)	13
Presentación	13
Migración de información entre bases de datos	13
Exportación de órdenes del día	14
Exportación del historial de órdenes	14
Exportación de facturas del día	14
Importación en la base destino	14
Uso de tablas temporales	15
Ajuste de secuencias	15
Dificultades encontradas	16
Procedimiento final	16
Conclusión	17

Introducción

El presente proyecto tiene como finalidad diseñar e implementar una base de datos para la administración de un restaurante. La solución propuesta permite organizar la información relacionada con empleados, clientes, productos, órdenes, facturas y el historial de consumo, facilitando el control de las operaciones diarias del negocio.

El problema principal consiste en almacenar y relacionar correctamente la información del restaurante, evitando redundancia de datos y asegurando la integridad de la información mediante llaves primarias, llaves foráneas y restricciones. Para ello se desarrolló primero un Modelo Entidad-Relación (MER), a partir del cual se obtuvo el Modelo Relacional (MR), transformando las entidades, relaciones y atributos en tablas normalizadas.

Además del diseño de la base de datos, se implementaron elementos propios de PostgreSQL como funciones, vistas, índices, secuencias y disparadores. Estos permiten automatizar tareas importantes, como la generación del folio de una orden, el cálculo de subtotales, la actualización del total de una venta y la validación de disponibilidad de productos.

Finalmente, el proyecto también contempla un proceso de migración de información entre bases de datos. Para ello se utilizaron archivos CSV, separando la información fija de catálogo de la información transaccional registrada durante el día. Con esto se permite transferir datos desde una base origen hacia una base destino de forma ordenada, evitando duplicados y manteniendo la integridad de los registros.

Plan de trabajo

Para la gestión y desarrollo de este proyecto, el equipo adoptó un enfoque basado en metodologías ágiles, específicamente utilizando un marco de trabajo Kanban a través de la plataforma Trello. Esta decisión permitió tener un control visual del flujo de tareas (clasificadas en "Por hacer", "En progreso" y "Completado"), facilitando la colaboración remota y asegurando el cumplimiento de las entregas en tiempo y forma.

Fase de Orquestación (ETL)

Para resolver la migración diaria de datos, diseñamos un flujo automatizado mediante scripts de PostgreSQL. El proceso consiste en exportar la información de la base original a archivos `.csv` mediante el comando `\copy`, separando las tablas fijas (como el catálogo de empleados o el menú) de las tablas transaccionales (órdenes y facturas filtradas por el día actual). Posteriormente, en la base de datos destino, cargamos estos archivos utilizando tablas temporales y la cláusula `ON CONFLICT DO NOTHING` para evitar registros duplicados. Finalmente, el script ajusta dinámicamente las secuencias (`setval`) para garantizar que los nuevos registros mantengan la integridad de los identificadores.

Asignación de roles y contribuciones

Aunque muchas decisiones se discutieron en equipo, las actividades documentadas en nuestro tablero de Trello se repartieron y trabajaron en parejas de la siguiente manera para avanzar más rápido con el proyecto:

- **Cruz Ortega Mauricio:** Tomó la parte de diseñar con Ricardo el Modelo Entidad-Relación (MER) y programar la lógica de la base (vistas, funciones, triggers y el índice). De igual manera, en conjunto con José, realizaron la creación y estructuración de las tablas. Para la parte final, armó el script de orquestación (ETL) del proyecto junto con Ricardo.
- **Gómez Domingo José Antonio:** Se encargó de crear el Modelo Relacional (MR) en conjunto con Ángel. Hizo equipo con Mauricio para la creación y estructuración física de las tablas, y elaboró los scripts para las inserciones masivas trabajando nuevamente a la par con Ángel.
- **Hernández Muciño Angel Gabriel:** Trabajó directamente con José en la creación del Modelo Relacional (MR) y en el armado de los scripts de inserciones. De forma individual, realizó la tarea de definir y programar todas las llaves primarias, llaves foráneas y las reglas de validación (checks) de las tablas.
- **Mugartegui Vaca Ricardo:** Hizo equipo con Mauricio para diseñar el Modelo Entidad-Relación (MER) y para programar toda la lógica en SQL (vistas, funciones, triggers e índice). Por su cuenta, fue el encargado de realizar las pruebas de requisitos y configurar el manejo de excepciones. En la etapa final, trabajó con Mauricio en la configuración de la orquestación automatizada.

Diseño

En esta sección se detalla el proceso de abstracción, estructuración y modelado de la base de datos orientada a la digitalización operativa del restaurante. El diseño se aborda desde dos etapas fundamentales: el diseño conceptual mediante el Modelo Entidad-Relación (MER) y el diseño lógico a través de la especificación del Modelo Relacional (MR).

Diseño Conceptual: Modelo Entidad-Relación (MER)

El Modelo Entidad-Relación desarrollado permite representar las reglas de negocio y los requerimientos informáticos del establecimiento de forma semántica y abstracta.

- **Jerarquía del Personal (Especialización):** Se implementó una entidad principal denominada EMPLEADO que centraliza los atributos generales de los trabajadores: RFC, número o identificador de empleado, nombre completo (descompuesto en nombre y apellidos), fecha de nacimiento, edad, sueldo, fotografía y domicilio. Asimismo, se incluye el atributo multivalorado **telefonos**. A partir de esta entidad, se estructuró una jerarquía parcial y traslapada para clasificar los puestos específicos requeridos por el negocio: **cocinero** (con su especialidad), **mesero** (con su respectivo horario) y **administrativo** (con su rol asignado).
- **Entidades Débiles:** Para llevar el registro del núcleo familiar del personal, se integró la entidad DEPENDIENTE (compuesta por CURP, nombre completo y parentesco). Esta se vincula mediante una relación de dependencia de existencia con EMPLEADO bajo una cardinalidad de $1 : M$, indicando que un empleado puede tener múltiples dependientes registrados.
- **Estructura del Menú (Productos y Categorías):** Los platillos y bebidas se unificaron bajo la entidad PRODUCTO (almacenando nombre, descripción, receta, precio,

indicador de disponibilidad y tipo de producto). Cada elemento se asocia de forma determinista a una sola CATEGORIA mediante una relación con cardinalidad $M : 1$, respetando la regla del negocio de pertenencia exclusiva a una categoría.

- **Flujo Operativo de Órdenes (Venta):** El proceso comercial inicia cuando un **mesero** registra una **orden**. La entidad **orden** resguarda el folio consecutivo de control, la fecha con hora de levantamiento y el monto total de la transacción. Para desglosar el consumo de forma detallada, la orden se conecta con la entidad asociativa **HISTORIAL_ORDEN** bajo una cardinalidad $1 : M$, la cual registra el identificador de detalle, la cantidad de unidades solicitadas, el precio unitario del momento y el subtotal por producto.
- **Módulo de Facturación y Clientes:** En caso de que el consumo requiera la emisión de un comprobante fiscal, la entidad **orden** genera una única **FACTURA** (cardinalidad $1 : 1$). Dicha factura se vincula de manera directa con un **CLIENTE** esquematizado por su RFC, nombre completo, domicilio fiscal, razón social, correo electrónico y fecha de nacimiento.

Diseño Lógico: Modelo Relacional (MR)

El diseño lógico traduce las abstracciones del MER en esquemas relacionales normalizados, garantizando la integridad referencial y el cumplimiento de las restricciones atómicas definidas por el problema.

- **Atomización de Atributos Compuestos:** Para satisfacer los principios de diseño y la primera forma normal, los atributos complejos de dirección y datos nominales se desagregaron completamente en campos atómicos independientes (**ESTADO**, **CODIGO_POSTAL**, **COLONIA**, **CALLE**, **NUMERO**, **NOMBRE**, **APELLIDO_PATERO** y **APELLIDO_MATERO**) dentro de las tablas **EMPLEADO** y **CLIENTE**.
- **Normalización de Atributos Multivalorados:** Con el fin de evitar la redundancia y grupos repetitivos, los números de contacto del personal se extrajeron de la tabla principal para conformar la relación subordinada **TELEFONO**, vinculada fuertemente mediante la llave foránea **ID_EMPLEADO**.
- **Transformación de la Herencia:** La especialización de roles se modeló mediante el enfoque de tablas independientes para cada subtipo (**COCINERO**, **MESERO**, **ADMINISTRATIVO**). En esta solución, el campo **ID_EMPLEADO** funge simultáneamente como Llave Primaria (PK) en su propia relación y como Llave Foránea (FK) refiriéndose a la tabla base **EMPLEADO**, asegurando que no existan registros huérfanos ni dispersión innecesaria de valores nulos.

Nota de Adaptación Arquitectural

Es importante hacer constar que, bajo las directrices de diseño establecidas para el desarrollo de esta solución técnica, la estructuración lógica de los esquemas, las restricciones de integridad y la selección explícita de los tipos de datos nativos presentes en el diagrama relacional (tales como las especificaciones de columnas basadas en dominios **VARCHAR2**, **NUMBER**, **CLOB** y **BLOB**) han sido adaptadas intencionalmente para corresponder de manera nativa con la arquitectura y el motor de almacenamiento de **Oracle Database**.

Implementación

Lenguaje de Definición de Datos (DDL) - Creación de la Base de datos y tablas

Primeramente, se crea una base de datos la cual tendrá toda la información necesaria para la administración del restaurante.

1. Para crear el entorno en donde se alojara la base se ocupa el siguiente fragmento.
Nota: Si se utiliza pgAdmin se debe de ejecutar primero este bloque y luego conectarse a la base manualmente antes de crear las tablas.

```
CREATE DATABASE BD_RESTAURANTE
WITH -- Esto junto al CREATE me permiten crear la base
OWNER = postgres -- Esto define que usuario es el administrador
de la base
ENCODING = 'UTF8' --Asi como en varios lenguajes de
estructuración, esto ayuda a la base a aceptar acentos y
otros caracteres.
CONNECTION LIMIT = -1; --Indica que no hay un limite de
usuarios permitidos a conectarse
```

2. Si se usa la terminal psql se debe retirar de comentar el siguiente fragmento el cual permite conectarse a la base creada.

```
--\c BD_RESTAURANTE
```

Para garantizar una ejecución fluida y libre de conflictos de dependencias en la creación de la base de datos en el sistema gestor PostgreSQL, el diseño implementado fue de orden jerárquico, dividiéndose en tres bloques principales:

- **Tablas Padre:** Son las tablas sin dependencias a otras tablas. (empleado, cliente y categoría).

```
CREATE TABLE empleado (
    id_empleado SERIAL,
    rfc VARCHAR(13) NOT NULL,
    nombre VARCHAR(50) NOT NULL,
    apellido_paterno VARCHAR(50) NOT NULL,
    apellido_materno VARCHAR(50),
    fecha_nacimiento DATE NOT NULL,
    edad INT CHECK (edad >= 18),
    sueldo NUMERIC(10,2) CHECK (sueldo >= 0),
    foto BYTEA,
    estado VARCHAR(50),
    codigo_postal VARCHAR(10),
    colonia VARCHAR(80),
    calle VARCHAR(80),
    numero VARCHAR(10)
);

CREATE TABLE categoria (
    id_categoria SERIAL,
    nombre VARCHAR(80) NOT NULL,
    descripcion TEXT
```

```
);

CREATE TABLE cliente (
    rfc_cliente VARCHAR(13),
    nombre VARCHAR(50) NOT NULL,
    apellido_paterno VARCHAR(50) NOT NULL,
    apellido_materno VARCHAR(50),
    razon_social VARCHAR(100),
    email VARCHAR(100),
    fecha_nacimiento DATE,
    estado VARCHAR(50),
    codigo_postal VARCHAR(10),
    colonia VARCHAR(80),
    calle VARCHAR(80),
    numero VARCHAR(10)
);
```

- **Tablas Hijas:** Son aquellas tablas que dependen de tablas padre mediante relaciones de herencia o lógica. (cocinero, mesero, administrativo y producto).

```
CREATE TABLE cocinero (
    id_empleado INT,
    especialidad VARCHAR(80) NOT NULL
);

CREATE TABLE mesero (
    id_empleado INT,
    horario VARCHAR(80) NOT NULL
);

CREATE TABLE administrativo (
    id_empleado INT,
    rol VARCHAR(80) NOT NULL
);

CREATE TABLE producto (
    id_producto SERIAL,
    id_categoria INT NOT NULL,
    nombre VARCHAR(80) NOT NULL,
    descripcion TEXT,
    receta TEXT,
    precio NUMERIC(10,2) NOT NULL CHECK (precio >= 0),
    disponibilidad BOOLEAN NOT NULL DEFAULT TRUE,
    tipo_producto VARCHAR(10) NOT NULL
);
```

- **Tablas Transaccionales:** Son esas tablas que dependen de múltiples tablas y son muy volátiles y registran la operación diaria del restaurante. Cuentan con múltiples llaves foráneas simultáneamente. (orden, factura e historial_orden).

```
CREATE TABLE orden (
    folio_orden VARCHAR(10),
    fecha_hora TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```

        total NUMERIC(10,2) NOT NULL DEFAULT 0,
        id_mesero INT NOT NULL
    );

    CREATE TABLE historial_orden (
        id_detalle SERIAL,
        folio_orden VARCHAR(10) NOT NULL,
        id_producto INT NOT NULL,
        cantidad INT NOT NULL CHECK (cantidad > 0),
        precio_unitario NUMERIC(10,2) NOT NULL CHECK (
            precio_unitario >= 0),
        subtotal NUMERIC(10,2) NOT NULL DEFAULT 0
    );

    CREATE TABLE factura (
        id_factura SERIAL,
        folio_orden VARCHAR(10) NOT NULL,
        rfc_cliente VARCHAR(13) NOT NULL,
        fecha_factura TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
    );

```

- **Tablas derivadas o secundarias** Son aquellas tablas que se crean a partir de atributos multivaluados o entidades débiles.

```

    CREATE TABLE telefono (
        id_telefono SERIAL,
        id_empleado INT NOT NULL,
        telefono VARCHAR(15) NOT NULL
    );

    CREATE TABLE dependiente (
        curp VARCHAR(18) NOT NULL,
        id_empleado INT NOT NULL,
        nombre VARCHAR(50) NOT NULL,
        apellido_paterno VARCHAR(50) NOT NULL,
        apellido_materno VARCHAR(50),
        parentesco VARCHAR(40) NOT NULL
    );

```

Asimismo, la integridad y consistencia de la información se ha blindado a nivel de esquema mediante el uso de restricciones explícitas de llave primaria (PRIMARY KEY), integridad referencial (FOREIGN KEY con reglas de eliminación/actualización), restricciones de dominio (CHECK para mitigar valores erróneos en tipos de producto o sueldos) y automatización de identificadores únicos mediante el tipo de dato SERIAL.

```

    ALTER TABLE producto ADD CONSTRAINT chk_tipo_producto CHECK (
        tipo_producto IN ('PLATILLO', 'BEBIDA'));

    ALTER TABLE empleado ADD CONSTRAINT pk_empleadoo PRIMARY KEY (
        id_empleado);

    ALTER TABLE cocinero ADD CONSTRAINT pk_cocinero PRIMARY KEY (
        id_empleado);

    ALTER TABLE mesero ADD CONSTRAINT pk_mesero PRIMARY KEY (id_empleado
    );

```



```

ALTER TABLE administrativo ADD CONSTRAINT pk_administrativo PRIMARY
KEY (id_empleado);

ALTER TABLE telefono ADD CONSTRAINT pk_telefono PRIMARY KEY (
id_telefono);

ALTER TABLE dependiente ADD CONSTRAINT pk_dependiente PRIMARY KEY (
id_empleado, curp);

ALTER TABLE categoria ADD CONSTRAINT pk_categoria PRIMARY KEY (
id_categoria);

ALTER TABLE producto ADD CONSTRAINT pk_producto PRIMARY KEY (
id_producto);

ALTER TABLE cliente ADD CONSTRAINT pk_cliente PRIMARY KEY (
rfc_cliente);

ALTER TABLE orden ADD CONSTRAINT pk_orden PRIMARY KEY (folio_orden);

ALTER TABLE historial_orden ADD CONSTRAINT pk_historial_orden
PRIMARY KEY (id_detalle);

ALTER TABLE factura ADD CONSTRAINT pk_factura PRIMARY KEY (
id_factura);

-- fk creacion de llaves foraneas
ALTER TABLE cocinero ADD CONSTRAINT fk_cocinero_empleado
FOREIGN KEY (id_empleado) REFERENCES empleado(id_empleado);

ALTER TABLE mesero ADD CONSTRAINT fk_mesero_empleado
FOREIGN KEY (id_empleado) REFERENCES empleado(id_empleado);

ALTER TABLE administrativo ADD CONSTRAINT fk_administrativo_empleado
FOREIGN KEY (id_empleado) REFERENCES empleado(id_empleado);

ALTER TABLE telefono ADD CONSTRAINT fk_telefono_empleado
FOREIGN KEY (id_empleado) REFERENCES empleado(id_empleado);

ALTER TABLE dependiente ADD CONSTRAINT fk_dependiente_empleado
FOREIGN KEY (id_empleado) REFERENCES empleado(id_empleado);

ALTER TABLE producto ADD CONSTRAINT fk_producto_categoria
FOREIGN KEY (id_categoria) REFERENCES categoria(id_categoria);

ALTER TABLE orden ADD CONSTRAINT fk_orden_mesero
FOREIGN KEY (id_mesero) REFERENCES mesero(id_empleado);

ALTER TABLE historial_orden ADD CONSTRAINT fk_historial_orden
FOREIGN KEY (folio_orden) REFERENCES orden(folio_orden);

ALTER TABLE historial_orden ADD CONSTRAINT fk_historial_producto
FOREIGN KEY (id_producto) REFERENCES producto(id_producto);

ALTER TABLE factura ADD CONSTRAINT fk_factura_orden
FOREIGN KEY (folio_orden) REFERENCES orden(folio_orden);

ALTER TABLE factura ADD CONSTRAINT fk_factura_cliente

```

```

FOREIGN KEY (rfc_cliente) REFERENCES cliente(rfc_cliente);

-- unique
ALTER TABLE empleado ADD CONSTRAINT uq_empleado_rfc UNIQUE (rfc);

ALTER TABLE factura ADD CONSTRAINT uq_factura_orden UNIQUE (
folio_orden);

```

Vistas computadas (Views)

Para optimizar las consultas recurrentes y concentrar la complejidad de las uniones (JOINS) entre múltiples tablas, se implementaron vistas computadas. Por lo que se destaca la generación de una vista simuladora de facturas, la cual permite encapsular automáticamente la información del cliente, la orden y el desglose de productos. Además, se crearon vistas analíticas para identificar rápidamente los productos agotados y determinar el platillo más vendido mediante el uso de funciones de agregación.

- Platillo mas vendido

```

-- PLATILLO MAS VENDIDO
CREATE OR REPLACE VIEW vista_platillo_mas_vendido AS
SELECT p.*, SUM(h.cantidad) AS total_vendido
FROM producto p
JOIN historial_orden h
ON p.id_producto = h.id_producto
WHERE p.tipo_producto = 'PLATILLO'
GROUP BY p.id_producto
ORDER BY total_vendido DESC
LIMIT 1;

```

- Simulación factura

```

-- SIMULA FACTURA
CREATE OR REPLACE VIEW vista_factura_orden AS
SELECT
    f.id_factura,
    f.fecha_factura,
    c.rfc_cliente,
    c.nombre || ' ' || c.apellido_paterno AS cliente,
    c.razon_social,
    o.folio_orden,
    o.fecha_hora AS fecha_consumo,
    p.nombre AS producto,
    h.cantidad,
    h.precio_unitario,
    h.subtotal,
    o.total AS total_orden
FROM factura f
JOIN cliente c ON f.rfc_cliente = c.rfc_cliente
JOIN orden o ON f.folio_orden = o.folio_orden
JOIN historial_orden h ON o.folio_orden = h.folio_orden
JOIN producto p ON h.id_producto = p.id_producto;

```

- Nombre de productos no disponibles

```
-- PRODUCTOS NO DISPONIBLES
CREATE OR REPLACE VIEW vista_productos_agotados AS
SELECT nombre
FROM producto
WHERE disponibilidad = FALSE;
```

Funciones (Functions)

Se desarrollaron funciones en PL/pgSQL para la generación de reportes paramétricos. Estas incluyen la consulta de la cantidad de ventas y montos recaudados los cuales son filtrados por un periodo de tiempo determinado. De igual manera, se implementó un reporte que permite mostrar la cantidad de órdenes diarias por mesero, el cual incorpora un manejo de excepciones (RAISE EXCEPTION) para negar la consulta de manera segura si el número de empleado ingresado no corresponde a dicho rol.

- Ventas por mesero

Dado el ID de un empleado, mostrar la cantidad y el total pagado por cada orden. En caso de no ser mesero, mostrar un mensaje de error.

```
-- VENTAS POR MESERO
CREATE OR REPLACE FUNCTION reporte_diario_mesero(p_id_empleado
INT)
RETURNS TABLE (cantidad_ordenes BIGINT, total_recaudado NUMERIC
) AS $$
DECLARE
    val_mesero BOOLEAN;
BEGIN
    -- Validar si el empleado es mesero
    SELECT EXISTS(SELECT 1 FROM mesero WHERE id_empleado =
p_id_empleado) INTO val_mesero;

    IF NOT val_mesero THEN
        RAISE EXCEPTION 'Error: El empleado no tiene el puesto
de mesero.';
    END IF;

    -- Retornar la consulta filtrada al dia actual
    RETURN QUERY
    SELECT COUNT(folio_orden), COALESCE(SUM(total), 0)
    FROM orden
    WHERE id_mesero = p_id_empleado AND DATE(fecha_hora) =
CURRENT_DATE;
END;
$$ LANGUAGE plpgsql;
```

- Ventas por periodo

Dada una fecha de inicio o de final,, mostrar el número de ventas y el monto total de ventas en ese periodo de tiempo.

```
-- VENTAS POR PERIODO
CREATE OR REPLACE FUNCTION reporte_ventas_periodo(fecha_inicio
DATE, fecha_fin DATE DEFAULT NULL)
RETURNS TABLE (numero_ventas BIGINT, monto_total NUMERIC) AS $$
BEGIN
```

```

-- Si no se proporciona fecha_fin, se asume que es de un
  solo dia
IF fecha_fin IS NULL THEN
  fecha_fin := fecha_inicio;
END IF;

RETURN QUERY
SELECT COUNT(folio_orden), COALESCE(SUM(total), 0)
FROM orden
WHERE DATE(fecha_hora) BETWEEN fecha_inicio AND fecha_fin;
END;
$$ LANGUAGE plpgsql;

```

Disparadores (Triggers)

La lógica de negocio transaccional y los cálculos matemáticos se automatizaron directamente en el motor de la base de datos mediante disparadores. Se programó un trigger para la asignación automática y formato del folio secuencial en las nuevas órdenes. A nivel de detalle, se implementaron validadores que verifican la disponibilidad del producto en tiempo real antes de permitir la inserción, calculan el subtotal de manera independiente y actualizan dinámicamente el monto total de la orden principal frente a cualquier evento de inserción, actualización o eliminación (CRUD).

- Disparador de prefijo .°RD-”

El disparador se creó con el fin de proporcionar un prefijo al identificador de cada orden.

```

-- SECUENCIA PREFIJO 'ORD-001'
CREATE SEQUENCE seq_folio_orden START 1;

-- Funcion del trigger
CREATE OR REPLACE FUNCTION generar_folio_orden()
RETURNS TRIGGER AS $$
BEGIN
  -- Concatena 'ORD-' con el numero secuencial relleno con
  -- ceros (LPAD)
  IF NEW.folio_orden IS NULL THEN
    NEW.folio_orden := 'ORD-' || LPAD(nextval('
      seq_folio_orden')::TEXT, 3, '0');
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Disparador
CREATE TRIGGER trg_generar_folio
BEFORE INSERT ON orden
FOR EACH ROW EXECUTE FUNCTION generar_folio_orden();

```

- Disparador de actualización de producto

Este disparador actualiza los totales de producto, validando que el producto siga disponible.

```

-- TRIGGER ACTUALIZACION PRODUCTO

```

```

-- Funcion del trigger
CREATE OR REPLACE FUNCTION validar_y_calcular_detalle()
RETURNS TRIGGER AS $$
DECLARE
    disp BOOLEAN;
    val_precio NUMERIC(10,2);
BEGIN
    -- Validar disponibilidad y precio
    SELECT disponibilidad, precio INTO disp, val_precio
    FROM producto
    WHERE id_producto = NEW.id_producto;

    IF disp IS NULL THEN
        RAISE EXCEPTION 'El producto no existe.';
    END IF;

    IF NOT disp THEN
        RAISE EXCEPTION 'El producto seleccionado no esta
        disponible en este momento.';
    END IF;

    -- Asignar el precio unitario del catalogo si no se envia
    explicitamente
    IF NEW.precio_unitario IS NULL THEN
        NEW.precio_unitario := val_precio;
    END IF;

    -- Calcular el subtotal por producto (cantidad * precio)
    NEW.subtotal := NEW.cantidad * NEW.precio_unitario;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Disparador
CREATE TRIGGER trg_validar_calcular_detalle
BEFORE INSERT ON historial_orden
FOR EACH ROW EXECUTE FUNCTION validar_y_calcular_detalle();

```

- Disparador de actualización de venta

```

-- Funcion del trigger
CREATE OR REPLACE FUNCTION actualizar_total_orden()
RETURNS TRIGGER AS $$
BEGIN
    -- Si es una insercion o actualizacion
    IF TG_OP = 'INSERT' OR TG_OP = 'UPDATE' THEN
        UPDATE orden
        SET total = (
            SELECT COALESCE(SUM(subtotal), 0)
            FROM historial_orden
            WHERE folio_orden = NEW.folio_orden
        )
        WHERE folio_orden = NEW.folio_orden;
        RETURN NEW;

    -- Si se elimina un detalle de la orden
    ELSIF TG_OP = 'DELETE' THEN

```

```

UPDATE orden
SET total = (
    SELECT COALESCE(SUM(subtotal), 0)
    FROM historial_orden
    WHERE folio_orden = OLD.folio_orden
)
WHERE folio_orden = OLD.folio_orden;
RETURN OLD;
END IF;
END;
$$ LANGUAGE plpgsql;

-- Disparador
CREATE TRIGGER trg_actualizar_total
AFTER INSERT OR UPDATE OR DELETE ON historial_orden
FOR EACH ROW EXECUTE FUNCTION actualizar_total_orden();

```

Índice (Index)

Se eligió un índice estándar sobre la columna `fecha_hora` en la tabla `Orden`. Dado que el sistema debe reportar ventas diarias de meseros y consultar totales por periodos de tiempo (fechas de inicio y fin), este índice optimizará de forma importante la velocidad de esas búsquedas, evitando que el DBMS revise toda la tabla al hacer los reportes.

```

-- INDICE
CREATE INDEX idx_orden_fecha ON orden(fecha_hora);

```

Presentación

Migración de información entre bases de datos

La segunda parte del proyecto consiste en transferir la información registrada durante el día desde una base de datos origen hacia una base de datos destino. Para realizar este proceso se utilizaron archivos CSV, ya que permiten exportar información desde PostgreSQL y posteriormente cargarla en otra base de datos.

Para realizar la migración se dividió la información en dos grupos: tablas de catálogo y tablas transaccionales.

Las tablas de catálogo contienen información que no depende directamente del día, por ejemplo: empleados, clientes, categorías y productos. Estas tablas se cargan principalmente al inicio o cuando exista algún cambio en sus registros.

Las tablas transaccionales contienen la información generada durante la operación diaria del restaurante. En este caso son:

- `orden`
- `historial_orden`
- `factura`

Estas tablas se exportan filtrando por la fecha actual, ya que representan la información registrada durante el día.

Exportación de órdenes del día

Para exportar las órdenes del día se utilizó el comando:

```
\copy (  
SELECT *  
FROM orden  
WHERE DATE(fecha_hora) = CURRENT_DATE  
) TO 'C:/Users/atlew/ProyectoBases/orden_dia.csv'  
WITH CSV HEADER;
```

La condición `DATE(fecha_hora) = CURRENT_DATE` permite obtener únicamente las órdenes registradas en el día actual.

La instrucción `WITH CSV HEADER` indica que el archivo se genera en formato CSV y que la primera fila contiene los nombres de las columnas.

Exportación del historial de órdenes

La tabla `historial_orden` no contiene directamente la fecha de la venta. Por eso se une con la tabla `orden`, ya que ahí se encuentra el campo `fecha_hora`.

```
\copy (  
SELECT h.*  
FROM historial_orden h  
INNER JOIN orden o  
ON h.folio_orden = o.folio_orden  
WHERE DATE(o.fecha_hora) = CURRENT_DATE  
) TO 'C:/Users/atlew/ProyectoBases/historial_orden_dia.csv'  
WITH CSV HEADER;
```

En esta consulta, `h.*` indica que solo se exportan las columnas de `historial_orden`. La tabla `orden` solo se utiliza para poder filtrar por fecha.

Exportación de facturas del día

Para exportar las facturas del día se utiliza una consulta similar:

```
\copy (  
SELECT f.*  
FROM factura f  
INNER JOIN orden o  
ON f.folio_orden = o.folio_orden  
WHERE DATE(o.fecha_hora) = CURRENT_DATE  
) TO 'C:/Users/atlew/ProyectoBases/factura_dia.csv'  
WITH CSV HEADER;
```

Con esto se exportan únicamente las facturas asociadas a órdenes realizadas durante el día actual.

Importación en la base destino

Antes de importar los archivos CSV, la base destino debe tener creada la misma estructura que la base origen. Para ello se utilizan archivos SQL separados:

- `Tablas.sql`: contiene tablas, llaves primarias, llaves foráneas y restricciones.

- `Programacion.sql`: contiene funciones, triggers, vistas, secuencias e índices.

Después de crear la estructura, los archivos CSV se importan a la base destino.

Uso de tablas temporales

Para evitar errores por datos duplicados, los archivos CSV no se cargan directamente en las tablas reales. Primero se cargan en tablas temporales y después se insertan en las tablas finales.

Ejemplo con la tabla `orden`:

```
CREATE TEMP TABLE tmp_orden AS
SELECT * FROM orden
WITH NO DATA;
```

Este comando crea una tabla temporal con la misma estructura que `orden`, pero sin datos.

Después se carga el archivo CSV en la tabla temporal:

```
\copy tmp_orden
FROM 'C:/Users/atlew/ProyectoBases/orden_dia.csv'
WITH CSV HEADER;
```

Finalmente, los datos se insertan en la tabla real:

```
INSERT INTO orden
SELECT *
FROM tmp_orden
ON CONFLICT DO NOTHING;
```

La instrucción `ON CONFLICT DO NOTHING` evita que se inserten registros repetidos. Si el registro ya existe, PostgreSQL lo ignora; si no existe, lo inserta.

Este mismo procedimiento se aplica para las demás tablas.

Ajuste de secuencias

Después de importar datos desde archivos CSV, es necesario ajustar las secuencias de las tablas que utilizan `SERIAL`. Esto se debe a que los identificadores se importan desde el archivo, pero las secuencias internas de PostgreSQL no se actualizan automáticamente.

Por ejemplo, si se importan empleados con identificadores del 1 al 5, la secuencia debe ajustarse para que el siguiente empleado tenga el identificador 6.

```
SELECT setval(
'empleado_id_empleado_seq',
COALESCE((SELECT MAX(id_empleado) FROM empleado), 1)
);
```

La función `MAX(id_empleado)` obtiene el identificador más alto existente. La función `COALESCE` evita errores si la tabla está vacía.

Las secuencias ajustadas fueron:


```

SELECT setval('empleado_id_empleado_seq',
COALESCE((SELECT MAX(id_empleado) FROM empleado), 1));

SELECT setval('telefono_id_telefono_seq',
COALESCE((SELECT MAX(id_telefono) FROM telefono), 1));

SELECT setval('categoria_id_categoria_seq',
COALESCE((SELECT MAX(id_categoria) FROM categoria), 1));

SELECT setval('producto_id_producto_seq',
COALESCE((SELECT MAX(id_producto) FROM producto), 1));

SELECT setval('historial_orden_id_detalle_seq',
COALESCE((SELECT MAX(id_detalle) FROM historial_orden), 1));

SELECT setval('factura_id_factura_seq',
COALESCE((SELECT MAX(id_factura) FROM factura), 1));

```

También se ajusta la secuencia encargada de generar los folios de las órdenes:

```

SELECT setval(
'seq_orden',
COALESCE(
    (SELECT MAX(CAST(SUBSTRING(folio_orden FROM 5) AS INT))
    FROM orden),
    1
)
);

```

En este caso, SUBSTRING(folio_orden FROM 5) obtiene la parte numérica del folio. Por ejemplo, de ORD-005 obtiene 005. Después, CAST convierte ese valor a número entero para ajustar correctamente la secuencia.

Dificultades encontradas

Una dificultad fue evitar la duplicación de datos. Si se utiliza directamente `\copy FROM` sobre una tabla real, PostgreSQL intenta insertar todos los registros del archivo CSV. Si algunos registros ya existen, se genera un error por clave primaria duplicada.

Para resolver este problema se utilizaron tablas temporales y la instrucción `ON CONFLICT DO NOTHING`. Con esto, los registros repetidos se ignoran y solo se insertan los nuevos.

Otra dificultad fue el manejo de las secuencias. Al importar datos desde CSV, los identificadores se copian con los valores originales, pero las secuencias no se actualizan solas. Por eso fue necesario usar `setval` para ajustarlas al valor máximo existente.

Procedimiento final

El procedimiento final para realizar la migración es:

1. Crear la base de datos destino.
2. Ejecutar el archivo `Tablas.sql`.
3. Ejecutar el archivo `Programacion.sql`.

4. Exportar las tablas de catálogo cuando sea necesario.
5. Exportar las tablas del día: `orden`, `historial_orden` y `factura`.
6. Crear tablas temporales en la base destino.
7. Cargar los archivos CSV en las tablas temporales.
8. Insertar los datos en las tablas reales con `ON CONFLICT DO NOTHING`.
9. Ajustar las secuencias con `setval`.
10. Verificar la información con consultas `SELECT`.

Con este procedimiento, la base destino puede recibir la información diaria de forma ordenada, evitando duplicados y manteniendo la integridad de los datos.

Conclusión

- **Cruz Ortega Mauricio:** este proyecto me permitió conocer de una forma más práctica cómo se diseña e implementa una base de datos para resolver una situación real, en este caso la administración de un restaurante. Se analizó el problema mediante el Modelo Entidad-Relación y después se transformó al Modelo Relacional para crear las tablas, relaciones y restricciones necesarias. También se implementaron funciones, vistas, índices y disparadores que ayudaron a automatizar procesos como el cálculo de totales y la generación de folios. La parte que presentó mayor dificultad fue la migración de la base de datos, ya que fue necesario cuidar que no se duplicaran registros, respetar las llaves foráneas y ajustar correctamente las secuencias después de importar la información. En general, el proyecto me ayudó a comprender mejor la importancia de un buen diseño y de una correcta implementación en PostgreSQL.
- **Gómez Domingo José Antonio:** A lo largo del proyecto nos enfrentamos a diversas dificultades. Uno de los principales retos fue hacer que el esquema físico (DDL) de la estructura de las tablas padre e hijo mantuvieran las relaciones correctas y no generara errores al momento de poblar las tablas con información, lo que nos obligó a ser sumamente cuidadosos con el orden jerárquico de las dependencias. De igual manera, la programación de los triggers requirió un buen análisis debido a que se tenía que manejar correctamente los distintos eventos transaccionales (INSERT, UPDATE, DELETE) sin generar errores de inconsistencias con lo desarrollado previamente. A pesar de estos retos, el proyecto se desarrolló correctamente pese a que se considera que el tiempo fue justo se logró realizar un entregable que cumpliera con los requisitos. El mayor logro fue encapsular la lógica de negocio como el cálculo de subtotales y validaciones de inventario directamente en PostgreSQL debido a que al pasar de programar de Oracle a PostgreSQL había ciertas confusiones en cuanto a ciertos comandos o declaraciones que si bien no eran tan significativas era importante identificar y modificar para poder desarrollar todo en el mismo entorno.
- **Hernández Muciño Ángel Gabriel:** Durante mi participación en el proyecto, uno de los principales retos fue definir correctamente la estructura de la base de datos para garantizar relaciones coherentes entre las tablas y mantener la integridad

de la información. Una de las dificultades presentadas consistió en establecer adecuadamente las restricciones y validaciones necesarias para prevenir inconsistencias o errores en los datos, lo que requirió prestar especial atención a la relación entre entidades y a las reglas del negocio. Como acierto, la correcta implementación de llaves primarias, llaves foráneas y restricciones de validación permitió construir una base de datos más sólida, consistente y funcional. De manera personal, este proyecto me permitió reforzar conocimientos sobre modelado relacional y comprender la importancia de planificar correctamente la estructura de una base de datos desde etapas tempranas, ya que esto influye directamente en el correcto funcionamiento, mantenimiento y escalabilidad del sistema.

- **Mugartegui Vaca Ricardo:** El proyecto realizado me permitió reforzar todos los conceptos adquiridos en el curso, diseñando desde cero la base de datos, empezando desde su Modelo Entidad-Relación, pasando por el Modelo relacional, su implementación en SQL, específicamente en PostgreSQL, aplicar programación para cumplir los requerimientos con funciones y disparadores. Uno de los retos mas grandes fue la creación de los disparadores dado que se manejaban muchos datos distribuidos en distintas tablas. También, el realizar la exportación de registros de la base de datos a otra, resultó un desafío dado que nunca había visto como se realizaba esa tarea.